Project Report: Architecting a Secure Cloud Vault on AWS

“Zero-Trust Security Hardening”

Author: Abdulrahman Abualenain

Date: April 2026

1. Introduction

This project demonstrates the implementation of a secure cloud storage solution using **Amazon S3** and **AWS IAM**. The primary goal was to establish a "Least Privilege" environment where a local machine could securely upload data to a specific cloud bucket without exposing the rest of the AWS infrastructure.

2. Technical Architecture

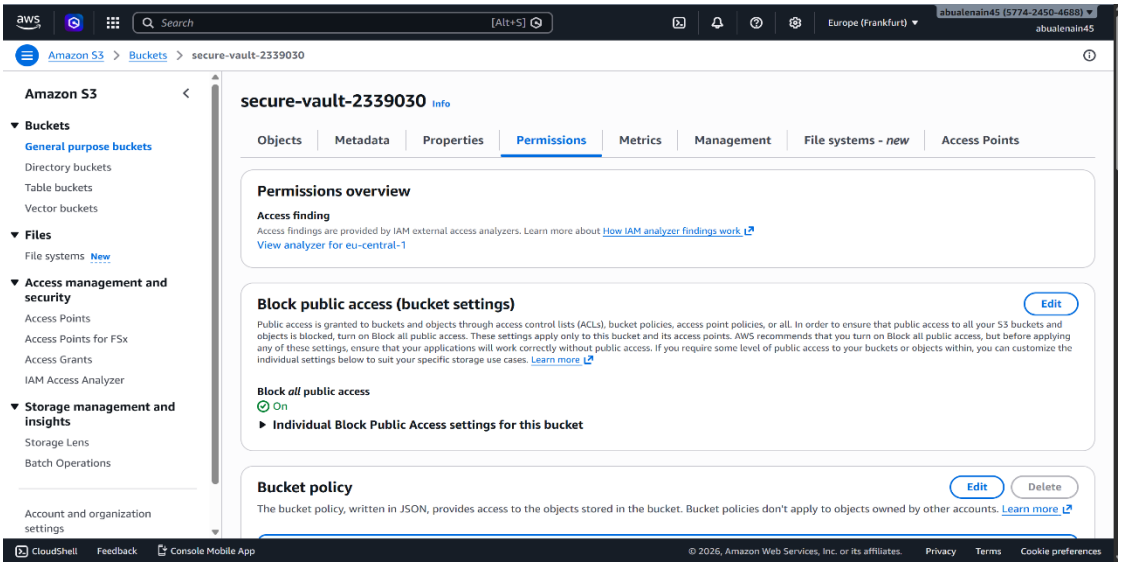
The solution uses a "Vault and Guard" model:

- **The Vault:** An S3 Bucket configured in the eu-central-1 (Frankfurt) region.
- **The Guard:** A dedicated IAM user with no console access, restricted to programmatic interactions via the AWS CLI.
- **The Bridge:** A custom-authored JSON policy ensuring that only authorized actions are permitted on the specific bucket resource.

3. Implementation Steps

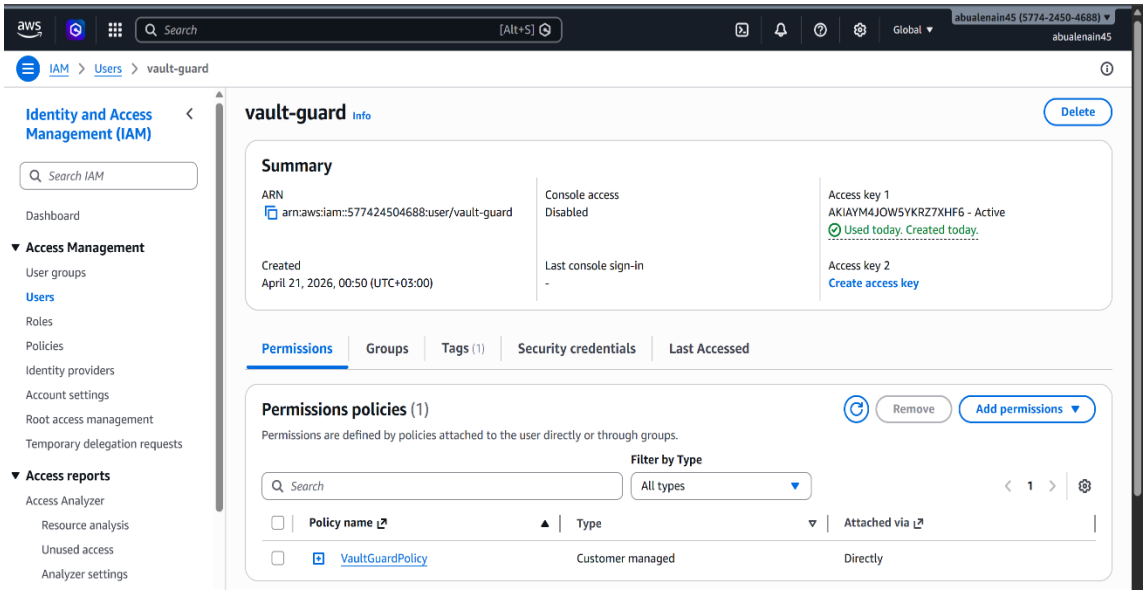
Phase 1: Storage Provisioning (S3)

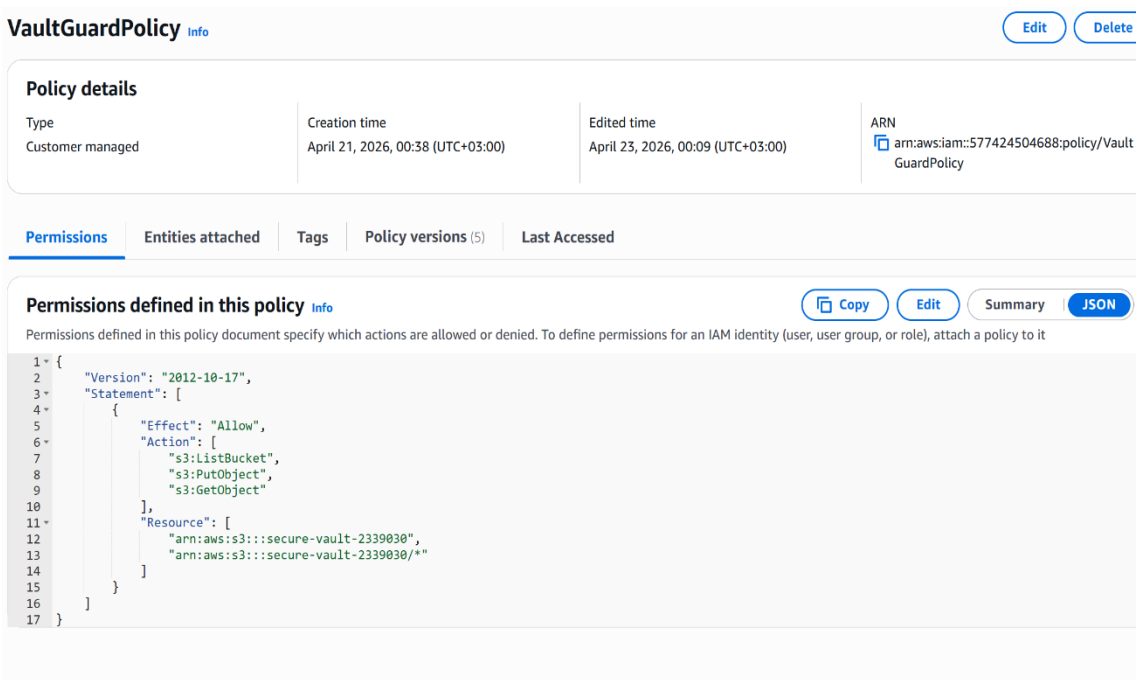
- Created a private S3 bucket named secure-vault-2339030.
- Ensured all "Public Access" was blocked to maintain a zero-trust posture.



Phase 2: Identity & Access Management (IAM)

- Created a new IAM user named vault-guard for programmatic access.
- **The Policy:** Authored a granular JSON policy. Unlike a "Full Access" policy, this specifically targets only the necessary bucket.





Phase 3: Local Environment Configuration

- Installed the **AWS CLI** on a Windows environment.
- Executed aws configure to link the local machine using secure **Access Keys**.
- Synchronized the region to eu-central-1 to match the bucket's physical location.

4. Problem Solving & Troubleshooting

A significant part of this project involved navigating the **Access Denied** challenges common in cloud security.

- **Issue:** Encountered an AccessDenied error when trying to list objects, even after creating a policy.
- **Diagnosis:** The policy was initially missing the **Bucket-level ARN**. In S3, permissions for the "folder" (arn:aws:s3:::bucket) and the "files" (arn:aws:s3:::bucket/*) are distinct.
- **Resolution:** I updated the JSON policy to include both resources and utilized wildcards for objects. This successfully implemented the **Principle of Least Privilege**.

```
C:\Users\Abdul>aws s3 cp final_test.txt s3://secure-vault-2339030/
upload failed: .\final_test.txt to s3://secure-vault-2339030/final_test.txt An error occurred (AccessDenied) when callin
g the PutObject operation: User: arn:aws:iam::577424504688:user/vault-guard is not authorized to perform: s3:PutObject o
n resource: "arn:aws:s3:::secure-vault-2339030/final_test.txt" because no identity-based policy allows the s3:PutObject
action
```

5. Verification of Success

The project was finalized by successfully "pushing" a local file to the cloud vault using the terminal.

- **Command used:** aws s3 cp final_test.txt s3://secure-vault-2339030/
- **Verification:** Confirmed the file was present in the AWS Console.


```
C:\Users\Abdul>aws s3 cp final_test.txt s3://secure-vault-2339030/
upload failed: .\final_test.txt to s3://secure-vault-2339030/final_test.txt An error occurred (AccessDenied) when callin
g the PutObject operation: User: arn:aws:iam::577424504688:user/vault-guard is not authorized to perform: s3:PutObject o
n resource: "arn:aws:s3:::secure-vault-2339030/final_test.txt" because no identity-based policy allows the s3:PutObject
action
```


6. Key Skills Demonstrated

- **Cloud Infrastructure:** S3 Bucket Management & Region Selection.
- **Identity Management:** IAM User creation and Access Key management.

- **Security Policy Authoring:** Writing and debugging JSON policies.
- **DevOps Tools:** AWS Command Line Interface (CLI) proficiency.